UVIC - McPHERSON
3 2775 90395981 4

QA76.5
F61



UNIVERSITY
OF VICTORIA
LIBRARY

COMPUTER MONOGRAPHS

General Editor: Stanley Gill, M.A., Ph.D., *Professor of Computing Science, Imperial College, London*
Associate Editor: J. J. Florentin, Ph.D., *Department of Computer Science, Imperial College, London*

7

AUTOMATIC SYNTACTIC ANALYSIS

AUTOMATIC SYNTACTIC ANALYSIS

J. M. FOSTER

MACDONALD: LONDON
AND
AMERICAN ELSEVIER INC.: NEW YORK

UNIVERSITY OF VICTORIA
LIBRARY

© J. M. Foster, 1970

First published 1970

Sole distributors for the United States and Dependencies
American Elsevier Publishing Company, Inc.
52 Vanderbilt Avenue
New York N.Y. 10017

Sole Distributors for the British Isles and Commonwealth
Macdonald & Co. (Publishers) Ltd
49–50 Poland Street
London W.1

All remaining areas
Elsevier Publishing Company
P.O. Box 211
Jan van Galenstraat 335
Amsterdam
The Netherlands

British SBN 356 02696 5
American SBN 444 19725 7
Library of Congress Catalog Card No. 73–92487

All Rights Reserved. No part of this publication may be reproduced, stored in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without the prior permission of the publishers

PRINTED AND BOUND IN ENGLAND BY
BALDING + MANSELL LTD
LONDON AND WISBECH

CONTENTS

1. Introduction 1

2. Context-free Grammars 5
- Nomenclature 5
- Examples of Grammars and their Properties 8

3. Parsing 12

4. Universal Parsing Methods 17
- Top-to-bottom Parsing 18
- *A priori* Rules for Top-to-bottom Parsing 22
- Bottom-to-top Parsing 24
- *A priori* Rules for Bottom-to-top Analysis 26
- A Global Parser 28
- Comparisons 30

5. Special Parsing Methods 31
- Bottom-to-top Methods 33
- Operator Precedence Grammars 34
- Precedence Grammars 40
- Transition Matrices 43

6. Transformations on Grammars 44
- Transformations to remove Left-recursion 46
- Other Transformations 48

7. Using Grammatical Analyses for Compilation 51
- Illegal Sentences 55

Appendices 56

References 63

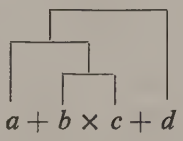
Index 65

1
INTRODUCTION

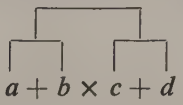
Frequently, in computing, we wish to deal with objects which have structure. The grammatical structure of an English sentence consists of subject, predicate, phrase and the like; a computer program has its structure of sub-program, procedure, expression; a line diagram has a structure of polygons and lines; a file of data has a structure determined by its index, and so on. We may wish to describe these structures so that we can deal with sets of them, like the set of grammatically correct English sentences, the set of structurally correct Algol programs or the set of properly formed files of some nature. These descriptions may be needed so that we can determine whether a given object is a member of such a set, or so that we can produce members of the set as examples, or so that we can manipulate the sets as a whole, as when we enquire which members are common to two sets. We may also be interested in the structure itself. For example, the grammatical structure of a sentence helps us to interpret it by revealing the relationships between the parts. Similarly, the structure of an Algol program tells us the proper relations between the parts, and is thus helpful if the program is to be compiled. Thus the structure of the Algol expression

$$a + b \times c + d$$

which can be indicated as



shows which parts of the arithmetic expression are related by the plus sign and which by the multiplication sign. If the structure had been



then the interpretation would have been totally different.

For these reasons we need formal descriptions for the structures of objects. Since the grammatical description of sentences was the first example studied, the nomenclature used is based on linguistic practice, even though we may be considering grammars for non-linguistic reasons. In this book we shall only be concerned with one-dimensional objects, consisting of sequences (sentences) of elementary items (words). By the structure (parse) of a sentence we shall mean the order and structure of its parts. We shall use the so-called context-free grammars. These are not the only ways of describing the structures of sentences, but they are the basis for most uses of grammars in compiling. [1, 7, 10, 13, 15]

We might be using grammars for two different reasons. We might wish to consider a grammar as a formal set of rules for generating all possible correct sentences of a language. In this case the problems are in making up grammars to generate languages, and in manipulating grammars. For example, the authors of the Algol report invented a grammar which would formally generate correct (grammatically) Algol programs. Alternatively we might be given a set of grammatical rules and a sentence, and be asked to parse the sentence according to the rules, or find whether there is a way of parsing it. The latter approach gives rise to problems in finding algorithms which will parse efficiently and can be guaranteed to terminate. These are the problems which will concern us in this book. Such algorithms are called parsers or syntax analysers.

The grammars that are used to describe computer languages are very much simpler than those that are needed for natural languages. Indeed the latter are by no means understood, and it is widely agreed that the kind of grammar which we shall be using is not satisfactory for natural languages.[8] Computer languages are designed to be easy to read and write (and compile), so they usually have a simple nature. Nevertheless, they are complex enough to present some difficulties, as we shall see. Context-free grammars used in the study of natural languages, on the other hand, yield grammars which are highly ambiguous; this fact points to their inadequacy as a complete description. It is normally necessary to find all the ways of parsing the given sentences. This tends to make the parsing processes for natural languages slow, whereas faster methods can be used to parse computer languages.

The problem of parsing is the problem of finding the proper places to insert the brackets which have been omitted. Consider the expression

$$a + b + c \times d \times e + f$$

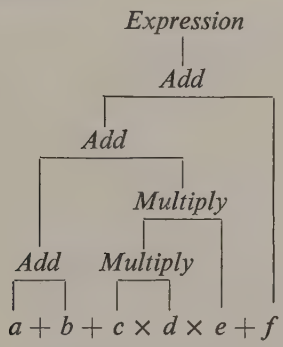
If this is written with brackets it becomes

$$(((a + b) + ((c \times d) \times e)) + f)$$

and the problem of parsing is trivial, because the brackets indicate the structure. However, Algol expressions or English sentences would be incomprehensible if brackets were inserted in this fashion, and so normally they are left out and only put in where vital. However, we might wish to distinguish different kinds of grouping. If the above expression were written with round brackets for the grouping associated with the plus sign and with square brackets for the multiplication sign it would appear like this

$$(((a + b) + [[c \times d] \times e]) + f)$$

Alternatively we could write



The account of parsing to be given in this book is directed towards the compilation of computer programs, but some of the problems of natural language parsing are touched on. No previous knowledge of grammars will be assumed, nor any knowledge of how compilers work, but some acquaintance will be assumed with computer programming. The algorithms for parsing will be discussed in a modified form of Algol, which should present no difficulty. In order to avoid cumbersome programs, list processing techniques have been used. A very brief account of these is given in Appendix 1.

Chapter 2 gives some elementary definitions and examples of grammars and Chapter 3 discusses the problems of parsing. Chapter 4 discusses methods of parsing applicable to all context-free grammars, and Chapter 5 special kinds of grammar developed for ease of parsing. Chapter 6 considers transformations on grammars, particularly those aimed at producing equivalent grammars which are easy to parse. Chapter 7, together with the first part of Chapter 6, shows very briefly how the structure of a computer program can be used to help in compilation.

Two early compilers systems based on parsing were those of Brooker and Morris, [3, 4, 5] and Irons.[21, 22] Further references may be found in an excellent review article by Feldman and Gries.[10]

Not all parts of a compiler are capable of being written using context-free grammars as a basis. Indeed the use of syntax analysis is usually confined to the first stages of a compiler, in which the structure of the program is extracted and some preliminary processing done. For example, it is not possible, using only context-free grammars, to impose such a restraint as 'all variables must be declared before being used' and this must be done during subsequent processing on the structure.

2

CONTEXT-FREE GRAMMARS

2.1 Nomenclature

We shall use the following nomenclature and notation. The set of properly formed sequences will be called *sentences*. They will consist of an ordered set of objects called *words*. We shall not concern ourselves with the structure of words, and we shall assume that we can recognise them in some unspecified way. The symbols *a*, *b*, *c*, *d*, *e*, *f*, +, × were the words in the example on page 3. Words will be in lower-case letters or will be symbols like + and ×. To help in the descriptions, we have a number of symbols, distinct from words, which we shall call *classes*. They are also called non-terminal symbols. In the preceding example *Add*, *Multiply* and *Expression* were classes. Classes will always be denoted by upper case letters. The sentence is a special class, for which the symbol *S* will usually be used. For each class there will be a number of possible alternative structures, each of which consists of an ordered set of items which are either words or classes. These are called *productions*. The whole grammar therefore consists of the set of words (the vocabulary), the set of classes, the name of the distinct class which is the sentence, and the set of productions.

To take a trivial example, consider the grammar with vocabulary *a*, *b*, *c* and *d*; with classes *S* and *T*; with sentence symbol *S* and with productions which we write as,

$$S \rightarrow a|b\,T\,T\,d$$
$$T \rightarrow b\,c|a$$

This means that there are two alternative structures for *S*: *a*, or *b* followed by *T* followed by *T* followed by *d*. Also two alternative structures exist for *T*. There are five legal sentences:

a
b b c b c d
b b c a d
b a b c d
b a a d

We shall say that these can be *derived* from S and write

$$S \rightarrow b\,a\,b\,c\,d$$

A part of the sentence which is derived from a class will be called a *phrase*. Thus in $b\ a\ b\ c\ d$ there is a phrase a derived from T and a phrase $b\ c$ also derived from T.

The productions can be considered as a set of rules for rewriting the classes, and the process of deriving $b\ a\ b\ c\ d$ from S can be considered as follows. Start with the sentence symbol.

$$S$$

and replace it by one of its alternatives

$$b\,T\,T\,d$$

Choose one of the classes in the result and rewrite that in turn with one of its alternatives

$$b\,T\,b\,c\,d$$

Continue this until there are no more classes left, in which case the sequence is said to be *terminal*. This happens after the next rewriting in this example.

$$b\,a\,b\,c\,d$$

We shall also say that $b\,T\,b\,c\,d$ is derived from S

$$S \rightarrow b\,T\,b\,c\,d$$

and that a is derived from T

$$T \rightarrow a$$

Note that it is a particular property of this kind of rewriting rule, that the substitutions used for a particular class in a particular place depend neither on the context in which it lies, nor on the sequence of rewriting operations from which it came. The class T can be replaced by a or by $b\ c$ without constraint. This is the origin of the name context-free which is applied to this kind of grammar. It means that one neighbourhood in a sentence cannot affect another distant neighbourhood, and is a strong constraint on the languages which can be described.

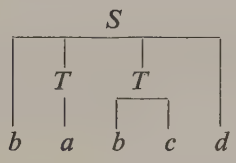
There are two orders in which we might have done the rewriting operations above

$$\begin{array}{l} S \\ b\,T\,T\,d \\ b\,T\,b\,c\,d \\ b\,a\,b\,c\,d \end{array}$$

or

$$
\begin{array}{l}
S \\
b\,T\,T\,d \\
b\,a\,T\,d \\
b\,a\,b\,c\,d
\end{array}
$$

These two ways of proceeding will not be considered as different parsings of *b a b c d*, nor will the grammar be considered ambiguous. Either we make the additional rule that we rewrite the leftmost class in the partially rewritten sequence; or we can say that we will consider the parse to be the structure

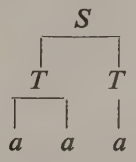


rather than follow the sequence of rewriting operations.

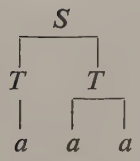
Grammars leading to ambiguous parsings can, however, easily be devised. The example

$$
\begin{array}{l}
S \rightarrow T\,T \\
T \rightarrow a \mid a\,a
\end{array}
$$

gives two parsings for the sentence *a a a*. These are



and



In computing languages ambiguity is to be avoided; the question of whether or not a grammar is ambiguous is an important one.

The choice of what is considered to be a word is arbitrary, provided that the words can be distinguished. For example, if a grammar is given which defines the set of sentences,

identifier
identifier + *identifier*
identifier + *identifier* + *identifier*

then we are at liberty to interpret *identifier* not as the particular sequence of letters *i*, *d*, *e*, *n*, *t*, *i*, *f*, *i*, *e*, *r* but as standing for members of a set of other objects, provided that they are distinguishable from +. Such a word will be called a *representative*. Use will be made of two representatives, *identifier* (abbreviated *id*) which stands for the set of sequences of letters and digits starting with a letter, and *number* which has its normal significance. An example of a legal sentence in the grammar above is

$$abc + d2 + xy3t$$

The grammars that we shall discuss will obey the following rules. There will be no derivations

$$X \rightarrow X$$

for any class X. Such derivations do not add to the set of sentences described, but they do allow the possibility of infinite regressions during parsing (an X is an X is an X . . .) Every class will occur in some derivation from S. Any class not occurring will not be used in the definition of the sentences, and may be dropped. Every class will have at least one derivation which is terminal. That is, there will be no classes like Y, where the only production for Y is

$$Y \rightarrow Ya$$

which never produces a terminal sequence.

2.2 Examples of Grammars and their Properties

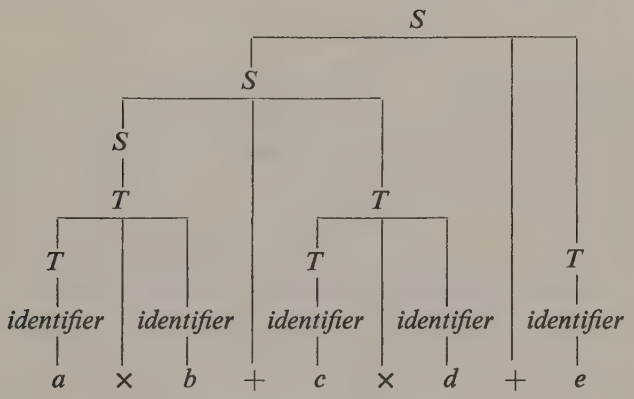
In this section various typical grammars will be discussed and the properties relevant to parsing described.

Grammars which define finite sets of legal sentences are of relatively little interest. The important cases are those in which the set of sentences is infinite. In order to define such infinite sets by the method

just described it is necessary to use recursion. This means that in some sequence of items (mixed words and classes) derived from a class X there is an occurrence of X. A typical case is the definition of the set of arithmetical expressions formed from identifiers and the signs $+$ and $\times$. A frequently used simple example where both S and T are recursive is

$$S \rightarrow T \mid S + T$$
$$T \rightarrow \textit{identifier} \mid T \times \textit{identifier}$$

A typical parsing of a sentence is



A class for which there is a derivation

$$X \rightarrow X\,\alpha$$

where α is a non-empty sequence of items, is called *left-recursive*; and both S and T are left-recursive classes. Similarly, a class for which there is a derivation

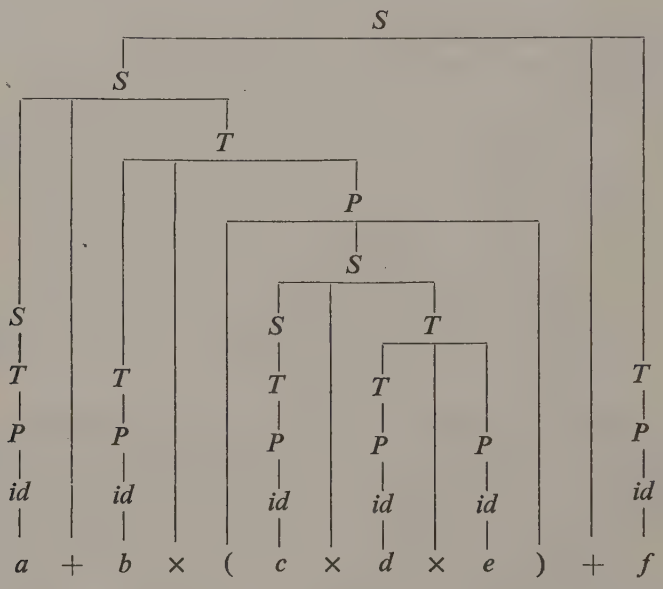
$$X \rightarrow \beta\,X$$

where β is a non-empty sequence of items, will be called *right-recursive*.

Another typical grammar is one for expressions involving identifiers, $+$, $\times$ and round brackets as follows,

$$S \rightarrow T | S + T$$
$$T \rightarrow P | T \times P$$
$$P \rightarrow id | (S)$$

where S, T and P are all recursive. A typical parsing of a sentence is



A class for which there is a derivation

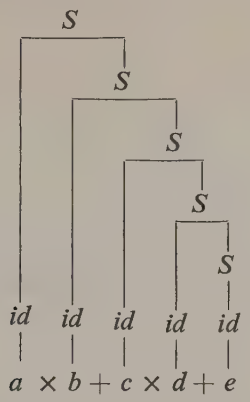
$$X \rightarrow \alpha X \beta$$

where α and β are non-empty sequences of items, is called *self-embedding*. The classes S, T and P are self-embedding, and S and T are left-recursive as well.

Two different grammars may well define the same set of sentences. For example, the set of expressions involving identifiers, $+$ and $\times$ previously defined, could also be defined by the grammar

$$S \rightarrow id | id + S | id \times S$$

A typical parsing using this grammar, however, is quite different:



If our aim is to compile the expression, the parsing given by the first grammar is useful since it indicates the arguments of the operators, but the second grammar is almost useless. This shows how we may have to choose one grammar rather than another because the parsing structure is more desirable. A way of partly freeing ourselves from this constraint is indicated in Chapter 6.

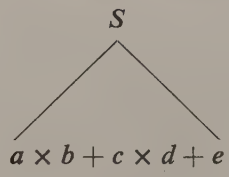
However the framework of context-free grammars is too wide, and grammars which are confusing can be defined. Consider

$$S \rightarrow a|a\ S\ a$$

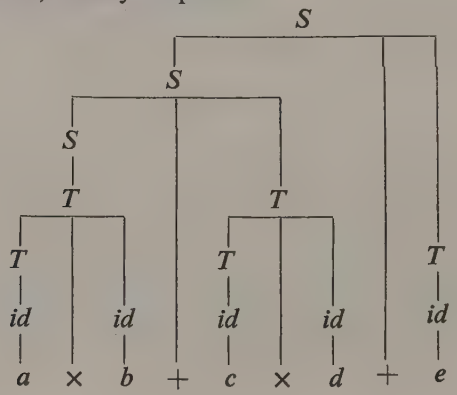
This grammar defines the set of sentences which are sequences of odd numbers of a, and specially distinguishes the middle one. As shown in the following chapter this is an awkward example to parse by some methods; also it is unlikely anyone would wish to do it. Various subsets of context-free grammars have been proposed, but none so far is wholly satisfactory. Hence parsing methods either are too general and inefficient, or the class of languages parsed is not wide enough.

3
PARSING

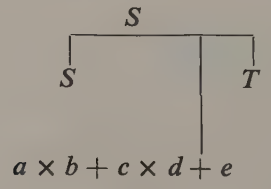
The problem of parsing can be regarded in the following way, which is based on a description given by Amarel[34]. Given, a particular sentence and a grammar, construct a triangle



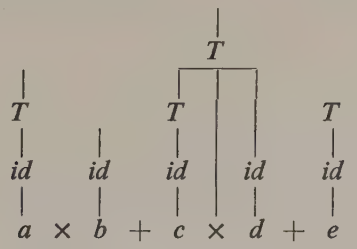
and attempt to fill the interior of the triangle with a self-consistent tree of derivations, namely the parse:



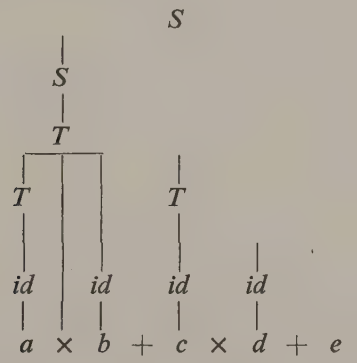
It is in principle unimportant how we attempt to fill the interior. We may do it from the top towards the bottom.



or from the bottom towards the top

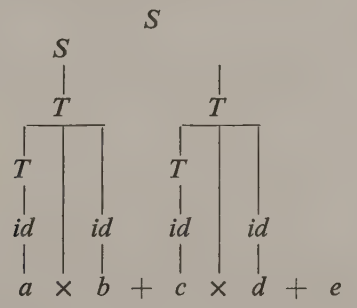


or outwards from the left corner



or indeed in any arbitrary order. The order, however, may greatly affect the efficiency of the process.

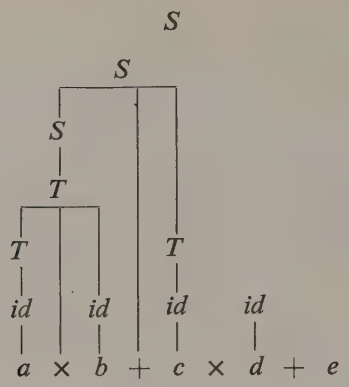
Whatever the order, the individual steps consist of trying to find a production to fit the locality under consideration. For example a possible procedure in the last diagram is to go to



by using the production,

$$T \rightarrow T \times id$$

This particular step fits the local situation and will also turn out to be correct in the total parse. We might, in the local situation have chosen the step



using the rule

$$S \rightarrow S + T$$

Later, we would have found it impossible to generate a self-consistent parse from this situation, and so we would have to come back and try a different local step. The fact that trial attempts have to be made which later turn out to be wrong, makes the parsing process potentially inefficient.

Let us consider the choice of the local step, regardless of the order in which we are trying to fill the triangle. We select a production, accepting it only if it passes some *a priori* tests. For example, we select a production only it it fits the local situation. The *a priori* tests can be much more elaborate. We might reject a production because it must necessarily lead to more words in the terminal sequence than are available; or we might reject it because it forces the use of a word which does not occur. If there were in the grammar under consideration a production,

$$S \rightarrow S - T$$

it would be useless to try to use it if the sentence did not contain a minus sign. We would know this without having to consider the exact structure. Another possible *a priori* rule for rejecting a production depends on the first word of the phrase which has to be derived from it. If the desired word cannot be derived as the first word, the production should be rejected. For example T must begin with an identifier.

After a production has been selected which cannot be faulted by the *a priori* rules, it is inserted and we carry on to the next stop. If we find later that the resulting structure cannot be made into a proper parsing of the sentence, then we shall have to return, select another production, and try again. If it is necessary to find all the parses, then we must try all the *a priori* satisfactory productions. If the language is known to be unambiguous, or if one parse is sufficient, then when we find a satisfactory parse the process can be terminated.

The speed of parsing depends on avoiding false trails, which cause work to be done which is later rejected. If the order of choosing the steps and the *a priori* rules is such that at each step only one production is possible, then the parsing method cannot be significantly improved (unless the *a priori* rules are very slow). But suppose that there are several choices for each step. Then the total number of possible parses increases exponentially. The choice of the *a priori* rules and the order of operating is therefore extremely important.

This can be illustrated by considering the grammar

$$S \rightarrow a|a\ S\ a$$

If this is parsed from top to bottom as follows,

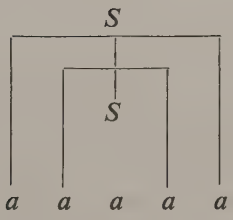
$$S$$

$$a\ a\ a\ a\ a$$

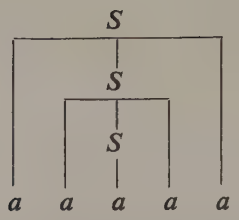
the first step is



the second step is



the third step is



We rely on the *a priori* rule that if the number of unattached words is one, then we use,

$$S \rightarrow a$$

and otherwise we use

$$S \rightarrow a\,S\,a$$

Thus there is only one way of proceeding. But if we parse from the left-hand corner, relying on the *a priori* rule that we only use a production which can start with the first unattached word (which is sometimes a useful rule, but in this case does not help), then we will have to follow false trials, and efficiency is lost.

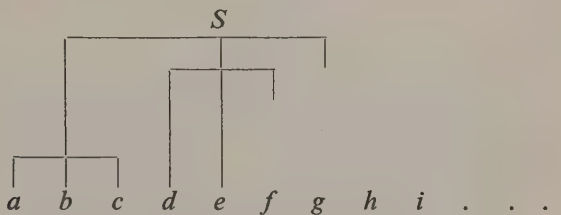
As well as the order in which the triangle is filled, and the *a priori* rules, we can choose the order in which the productions, not faulted by the *a priori* rules, are selected. This is not helpful unless the grammar is known to be unambiguous, or unless one parse is sufficient, because otherwise all the productions will have to be tried. But if one parse is sufficient, it may help to increase the probability of finding a parse by selecting the productions in some particular order.

4

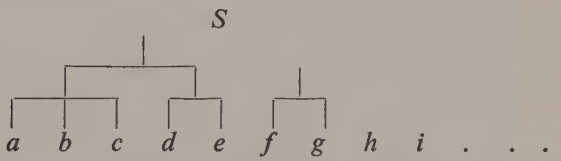
UNIVERSAL PARSING METHODS

This chapter discusses some of the methods of producing all the parses of a sentence belonging to a general context-free grammar.

Natural language sentences are usually fairly short, and they can be held in the store of the computer without difficulty; methods of parsing which require the simultaneous presence of the whole sentence can therefore be applied. This is not usually true of the sentences (programs) to be analysed by a compiler. Such sentences may consist of thousands of words. In such a case it is necessary to try to fill in the parse triangle in a way which requires only part of the sentence to be present at a time. An obvious way of doing this is to use the words, one at a time, from left to right in their natural sequence. Such a method leads to two fundamental ways of filling in the parsing triangle, from top-to-bottom and left-to-right



or from bottom-to-top and left-to-right



The main variety within these methods comes in the *a priori* rules which are used to limit the possible productions to be tried. The properties of top-to-bottom and bottom-to-top parsing algorithms are quite different.

Though a single attempted parse goes from left-to-right and either

upwards or downwards, several attempts may have to be made to parse a sequence of words, either because the inconsistency of a particular parse may only appear later in the sentence, or because the grammar is ambiguous. These attempted parses could all be carried out simultaneously, so that the sentence is processed steadily from left-to-right, or carried out sequentially, so that the sentence is scanned backwards for each failed or completed parse to pick up the starting point. The algorithms demonstrated below are of the simultaneous type.

4.1 Top-to-bottom Parsing

The top-to-bottom method starts with the sentence symbol and makes successive substitutions for classes to try to fit the sentence.

The nature of the algorithm can be illustrated by the following example. Consider the grammar

$$S \rightarrow T|T + S$$
$$T \rightarrow id|id \times T$$

where *id* is the representative for identifiers. We will first parse the sentence

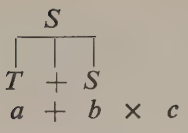
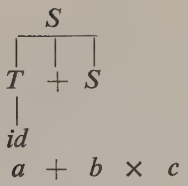
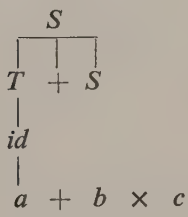
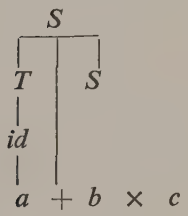
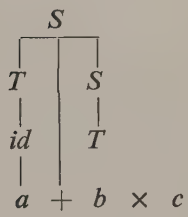
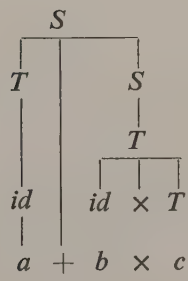
$$a + b \times c$$

choosing the productions which happen to be the correct ones and demonstrating the simultaneous creation of the parse tree. We will then show the full set of steps of the analysis process, leaving out the creation of the parse trees.

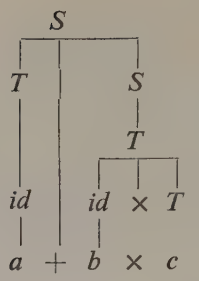
Start with S, and replace it by its alternatives, in this case $T + S$ is the correct one. In future steps, if the first symbol is a word, then it is compared with the first symbol of the sentence, and if they are the same, this symbol is deleted both from the sentence and the sequence of items. This sequence represents the permissible structure for the rest of the sentence. If the first symbol is a class, then it is replaced by its alternatives. Since we are choosing the correct productions the words will be the same, but in general they may not be, and if they are not the parse fails. The successive steps of the analysis are given below, showing the remainder of the sentence, the remainder of the parse, and the current parse tree at each stage:

$a + b \times c$ $\qquad$ S $\qquad$ S

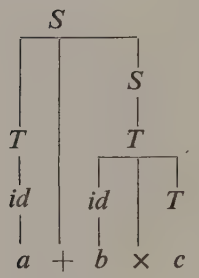
$a + b \times c$

$a + b \times c$	$T + S$	
$a + b \times c$	$id + S$	
$+ b \times c$	$+ S$	
$b \times c$	S	
$b \times c$	T	
$b \times c$	$id \times T$	

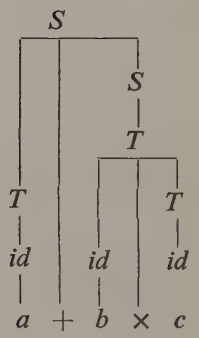
$\times\ c$ $\times\ T$

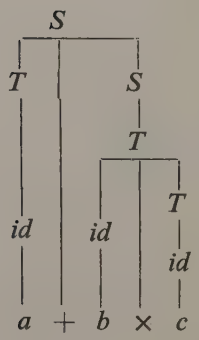


c T



c id





We now demonstrate all the alternatives showing the remainder of the sentence and the remainder of the structure, but omitting the constructed parses:

$a + b \times c$	S
$a + b \times c$	T
	$T + S$
$a + b \times c$	id
	$id + S$
	$id \times T$
	$id \times T + S$
$+ b \times c$	$-$
	$+ S$
	$\times T$
	$\times T + S$
$b \times c$	S
$b \times c$	T
	$T + S$
$b \times c$	id
	$id \times T$
	$id + S$
	$id \times T + S$
$\times c$	$-$
	$\times T$
	$+ S$
	$\times T + S$
c	T
	$T + S$
c	id
	$id \times T$
	$id + S$
	$id \times T + S$
$-$	$-$
	$\times T$
	$+ S$
	$\times T + S$

At the final step, the sentence and the first remainder are both empty, so this represents the correct parse.

An algorithm for top-to-bottom parsing is given in Appendix 2.

4.2 *A priori* Rules for Top-to-bottom Parsing

There is one big defect in the top-to-bottom method. Consider the grammar

$$S \rightarrow T|S + T$$
$$T \rightarrow id|T \times id$$

and attempt to parse with this. The successive substitutions will go

$$S$$

$$T$$
$$S + T$$

$$id$$
$$T \times id$$
$$T + T$$
$$S + T + T$$

and the process will never terminate. Hence the method will only work for grammars which contain no left-recursive classes, that is, no derivations

$$T \rightarrow T\,\alpha$$

It has been shown by Greibach[35] that it is always possible to transform a grammar into another equivalent one which is not left-recursive. An equivalent grammar is one which defines the same sentences and has the same ambiguities, but produces a different parse for a given sentence. Chapter 6 discusses transformations of this type, and also the problems of producing the correct parse from the transformed grammar.

There is an *a priori* rule which will remove the danger of looping in this fashion. Every class gives rise to a terminal sequence of words containing at least one word. Hence, if the number of items in the remainder parse is greater than the number of words left in the sentence, the parse cannot be correct. This rule is useful in practice only if the length of the sentence is small. It is therefore applicable, perhaps, in natural language processing, but it cannot be considered for compilation. In what follows we shall assume that the grammar contains no left-recursive classes.

A useful *a priori* rule can be devised which uses the current word of the sentence. Suppose that the current word is *b* and that we are about to replace the leftmost class, *X*, by its alternative productions. We need only select those which can yield phrases starting with *b*. If initially the grammar is processed to produce a matrix of words against classes, indicating which classes can begin with which words, then this matrix can be used to weed out the productions which cannot give rise to the starting word.

Such a matrix can easily be constructed. Suppose that we start with a matrix of words and classes against classes, indicating those words and classes which begin the productions for the classes. For example

$$S \rightarrow a|T\,b$$
$$T \rightarrow c|T\,d$$

gives

	S	*T*	*a*	*b*	*c*	*d*
S	0	1	1	0	0	0
T	0	1	0	0	1	0

indicating that *S* begins with *T* or *a*, and that *T* begins with *T* or *c*. Into the row for each class 'or' the rows of the classes present in that row, and repeat this until the whole matrix is stationary. Then the matrix indicates all the words which can begin the classes:

	S	*T*	*a*	*b*	*c*	*d*
S	0	1	1	0	1	0
T	0	1	0	0	1	0

An alternative way of obtaining a similar effect is to take the grammar (which contains no left-recursive classes) and repeatedly substitute the alternative productions for classes occurring at the left of productions, until all the productions begin with a word. For example.

$$S \rightarrow T|T + S$$
$$T \rightarrow id|id \times T$$

yields

$$S \rightarrow id|id \times T|id + S|id \times T + S$$
$$T \rightarrow id|id \times T$$

Then form, for each pair of class and word, the productions of the class which begin with the word. The *a priori* rule is to use the productions associated with the class at the start of the remainder of the parse, and the word at the start of the remainder of the sentence. This is identical in effect to the previous *a priori* rule in the absence of left-recursion. The expansion of the productions is done initially, instead

of during the parsing. It is therefore a little faster, but uses a little more store.

The transformed grammar with words at the start of all productions has been called 'in normal form', and the method of analysis is referred to as predictive analysis. Since the grammar may be somewhat different from the initial grammar, steps may have to be taken to recover the required parse.

A measure of economy of a different kind may be used. It is perhaps more useful for highly ambiguous grammars. If, in two or more separate alternative parses, it is found that the same sequence of words must be derived from the same sequence of classes and words, then the methods so far described would accomplish this parsing two or more times. This is because the alternative parses became separate at an earlier stage, and so the repeated work is not recognised. Where this might happen with any frequency it is desirable to do the analysis only once. It may also happen that the sequence of words does not fit the remainder of the parse. It is equally desirable not to establish this fact repeatedly. A method of avoiding repetition for an analyser intended for natural languages has been described by Kuno.[25]

4.3 Bottom-to-top Parsing

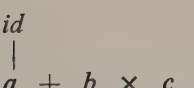
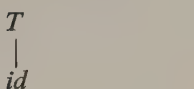
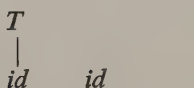
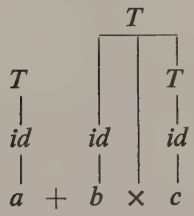
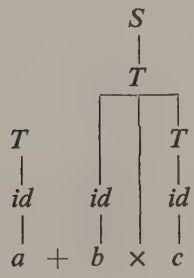
In opposition to the top-to-bottom method, in which by successive substitutions the sentence symbol is expanded to produce a sentence, the bottom-to-top method starts with the sentence and applies the productions backwards, trying to contract to the sentence symbol. Thus in the grammar

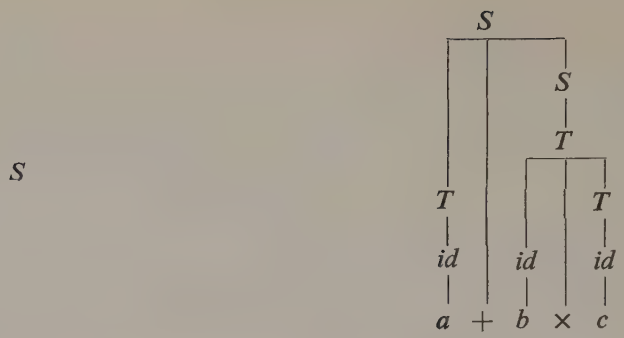
$$S \rightarrow T \mid T + S$$
$$T \rightarrow id \mid id \times T$$

we can analyse a sentence by the following steps

$a + b \times c$	
$id + id + id$	
$id + id \times T$	$T \rightarrow id$
$T + id \times T$	$T \rightarrow id$
$T + T$	$T \rightarrow id \times T$
$T + S$	$S \rightarrow T$
S	$S \rightarrow T + S$

The order in which the substitutions are made could be arbitrary, but since we are considering a scheme in which the sentence is processed from left to right, the substitutions must always be made as far to the left as possible. Once again we substitute the same sentence in the same grammar, proceeding from left to right, using the productions which we happen to know will be correct, and showing the formation of the parse tree:

$a + b \times c$	$a + b \times c$
$id + b \times c$	
	
$T + b \times c$	
	
$T + id \times c$	
	
$T + id \times id$	
	
$T + id \times T$	
$T + T$	
$T + S$	



4.4 *A priori* Rules for Bottom-to-top Analysis

Alone the bottom-to-top method is not particularly efficient because a large number of false trails may be followed. However, a powerful *a priori* rule exists.

Consider the above example at the stage,

$$id + b \times c$$

The only productions worth applying are those beginning with *id*. Each such production is part of the definition of an intermediate class. Unless that class can begin a sentence, there is no point in using the production. In this case two productions are applicable

$$T \rightarrow id$$
$$T \rightarrow id \times T$$

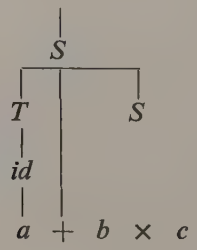
and since T can begin S, both must be tried. The second fails, leaving

$$T + b \times c$$

Once again, there are two applicable productions

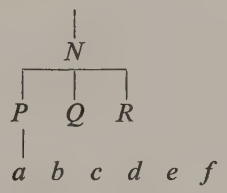
$$S \rightarrow T$$
$$S \rightarrow T + S$$

and both must be tried. The first fails. Now we know that the parse is



and therefore that the remainder has to start with an S. Therefore it is only worth applying productions to the start of $b \times c$ which define classes which can start S.

In this example productions are not eliminated by this rule, but in a more complex grammar they would be. In the example, the phrase $b \times c$ is to be parsed as an S, which happens to be the sentence symbol, but in general we parse the remainder to form any sequence of intermediate classes



and we apply only those productions which define classes which could start Q.

The formation of the matrix indicating which classes can start examples of other classes is exactly as described for the top-to-bottom method.

Consider the way in which this algorithm works when parsing the sentence $a + b \times c$, this time using the grammar

$$S \rightarrow T|S + T$$
$$T \rightarrow id|T \times id$$

to show that left-recursion is no problem. All the alternatives will be shown, but the formation of the parse tree will be ignored:

SENTENCE	GOAL	PRODUCTION
$a + b \times c$	$(S),S$	–
$a + b \times c$	$(id),T \quad (S),S$	$T \rightarrow id$

On the left is the sentence, on the right is the production which has been used, and in the middle is a sequence of pairs. The first member of each pair is the remainder of a production and the second member is the class for which this is a production. If the first member of the first production is equal to the first word of the sentence, then we remove it from both the sentence and the production:

SENTENCE	GOAL	PRODUCTION
$+ b \times c$	$NIL,T \quad (S),S$	–

If the first item in the parse sequence is empty, it is removed, and

productions are used which start with the corresponding class and define classes which can start the first class of the next pair. In this case we use the productions starting with T which define classes which can start S. The T is deleted from the production:

SENTENCE	GOAL	PRODUCTION
$+ b \times c$	$(x, id),T \quad (S),S$	$T \rightarrow T \times id$
	$NILS,S \quad (S),S$	$T \rightarrow T$

If the first symbol is a word, not identical to the first word of the remainder of the sentence, the parse fails. Thus the first alternative above fails. Expanding the second one we obtain

SENTENCE	GOAL	PRODUCTION
$+ b \times c$	–	–
	$(+, T),S \quad (S),S$	$S \rightarrow S + T$

The first alternative corresponds to the completion of the parse and therefore fails.

SENTENCE	GOAL	PRODUCTION
$b \times c$	$(T),S \ (S),S$	–
$b \times c$	$(id),T \ (T),S \ (S),S$	$T \rightarrow id$
$\times c$	$NIL,T \ (T),S \ (S),S$	–
$\times c$	–	–
	$(+, T),S \ (S),S$	$S \rightarrow S + T$
	$(\times, id),T \ (T),S \ (S),S$	$T \rightarrow T \times id$
c	$(id),T \ (T),S \ (S),S$	–
–	$NIL,T \ (T),S \ (S),S$	–
–	–	–
	$(+, T),S \ (S),S$	$S \rightarrow S + T$
	$(\times, id),T \ (T),S \ (S),S$	$T \rightarrow T \times id$

The first of these last three productions is the correct parse, and the rest fail.

4.5 A Global Parser

A parser based on top-to-bottom techniques which uses a number of *a priori* rules has been described by Unger.[32] This parser requires the presence of the whole sentence.

Whereas in the previous top-to-bottom parser the left end of a phrase and the name of a class is given but the right boundary is unknown, Unger's parser always knows the name of the class and both the left and right boundaries of the phrase which is to be

derived from it. This is true at the start, and successive steps maintain the feature.

Suppose that the sequence of words

$$a\,b\,c\,d\,e\,f\,g$$

is to be matched against the production

$$P \rightarrow a\,Q\,d\,e\,R$$

This can be correct only if $b\ c$ can be derived from Q and $f\,g$ can be derived from R. The algorithm is as follows. We are given a sequence of words, α, and a production, β, and we wish to show that they match. The production may contain some words. Examine α to see if these words occur in the correct order. If they do not, then this parse fails. If they do, select each of the different ways in which they occur in turn (if all the parses are wanted). The fragments of α between the words must be derived from the classes which occur between the same words in β. Here once again we have segments which must be derived from particular classes, and the matching algorithm can be called recursively. Two classes occurring next to each other in a production may give rise to a considerable number of alternatives.

Unger's parser uses further *a priori* rules to reject productions. Once the words have been found in α and a particular way of splitting α up has been chosen, four further tests are applied to the pieces before attempting to match any of them using the full algorithm.

First, the segment is checked to make sure that it has at least the minimum length needed for derivation from the class. A table of such minimum lengths is formed before starting to parse.

The second and third tests resemble the *a priori* rules for left-to-right analysis. The words at the left and right ends of the segment are checked to make sure that they can start and finish phrases derived from the class. This information can be tabulated as previously described.

In the fourth test, the segment is checked to make sure that it does not contain any words which cannot occur in phrases derived from the class. Once again, this information is tabulated.

Unger states that all these 'quick-checks' are needed for the efficiency of the parser to be satisfactory and that the resulting parser has compared well with other methods for a number of grammars.

Other *a priori* rules could be added. The parsing of a phrase need not take place by the full algorithm if a special parser can be written for the class in question. This is made easier because both boundaries of the phrase are known.

4.6 Comparisons

A general comparison of the parsing methods is difficult for two reasons. First, some grammars are parsed more efficiently by top-to-bottom methods and others by bottom-to-top methods, so the efficiency depends on the grammar. Second, if a particular grammar is found to be parsed inefficiently by one method, a transformed version of it can usually be found which can be parsed efficiently by that method. In practice it is found that for the simple grammars used in compilers a transformed version can be found to suit either method. However, the top-to-bottom left-to-right method has the defect that left-recursive classes have got to be removed to make a useful grammar.

Griffiths and Petrick[18] have made a comparison of parsers using an idealised machine (based on a Turing machine) and have found the number of steps needed by this machine to parse various grammars in various ways. They have also shown that there is always an equivalent grammar which can be parsed by a top-to-bottom method not much more slowly than the original was parsed by a bottom-to-top method.

Most parsers which produce all the parses have been based on top-to-bottom methods. Most of the special purpose parsers for restricted kinds of grammar can be thought of as based on bottom-to-top methods.

Of early systems, Brooker and Morris's compiler-compiler was based on top-to-bottom methods and Iron's PSYCHO was bottom-to-top.

5

SPECIAL PARSING METHODS

Methods of parsing which produce all analyses of any sentence in any context-free grammar are unlikely to be fast enough for use in compilers. Therefore, for fast compilation, parsing methods are usually used which will only produce one analysis and will only work for a restricted class of grammars. Such methods can be based on either top-to-bottom or bottom-to-top methods, and theyu sually work from left-to-right. The parsers and the class of grammars for which they work are usually chosen so that *a priori* rules exist which mean that only one production is applicable at each step, so that the parse can be directly generated, without having to back-track. This is not true for all such methods, in particular Iron's PSYCHO which only works for some grammars but allows back-tracking.

The first method to be described can be thought of as based either on top-to-bottom or on bottom-to-top parsing; the others are based on bottom-to-top parsing.

Consider top-to-bottom left-to-right analysis, with the same *a priori* rule as before; a production is used only if the next word of the sentence is one which can start an example of the production. If the grammar is such that for each class no word can start an example of more than one production, then the *a priori* rule indicates that there is only one possible production to apply at each step. For example

$$S \rightarrow a\ X$$
$$X \rightarrow b|c\ X$$

which defines the sentences

$$a\ b$$
$$a\ c\ b$$
$$\cdot$$
$$a\ c \ldots\ldots\ldots c\ b$$

can be parsed simply by noticing, while trying to match X, that when the remainder of the sentence starts with b then the first production is to be used, and when it starts with c the second production is the correct one.

It is easy to see whether a grammar is of this form, by constructing the matrix of possible starting words for each class and noting whether or not the sets of words starting the productions for a class are disjoint. However, it is unlikely that a given grammar will already have this property, so a transformation may have to be applied (Chapter 6).

When a grammar does have this property, the parser given in Appendix 2 can be modified to give a quick parse. But an even faster version can be made by translating the grammar itself into a program to do the parsing. A compiler writing system based on this has been described by the author.[14]

Suppose, as in the program in Appendix 2, that *symb* holds the next word of the sentence. Each class can be translated into a recursive procedure for recognising phrases derived from it. The previous grammar translates into

```
procedure S;
begin if symb = 'a' then fail;
      symb := next;
      X
end;

procedure X;
begin if symb = 'b' then complete
      else if symb = 'c' then begin symb := next;
                                    X
                              end
      else fail
end;

symb := next;
S
```

The first word is put into *symb* and *S* is called. If this word is not *a*, then the parse fails. If it is, then the next word is put into *symb* and *X* is called. *X* works in a similar fashion. This is a fast process, and indeed resembles the way in which an *ad hoc* recogniser might have been written, without knowledge of grammar.

So far, we have dealt entirely with grammars in which the productions are defined in terms of words and classes only. It is sometimes useful to introduce the empty production

$$T \rightarrow \phi$$

which means that *T* can be replaced by an empty sequence.

Using this, a grammar for arithmetic expressions involving

identifiers, + and × can be written which defines the same language as before, but obeys the rule required by this method:

$$S \rightarrow T\ X$$
$$X \rightarrow \phi \mid +\ T\ X$$
$$T \rightarrow id\ Y$$
$$Y \rightarrow \phi \mid \times\ id\ Y$$

Where ϕ is used, it is necessary to amplify the rule. If a class contains ϕ as a production, the sets of words which begin the non-empty productions must be distinct from each other and from the set of words which follow the class in the language.

The translation of the productions into program can be improved by noting that if a class is used in one place only, the procedure may be written out in that place. We can also replace procedure statements for the class at the end of a production by a jump:

```
procedure S;
begin na: if not id(symb) then fail;
          symb := next;
          if symb = '×' then begin symb := next;
                                   go to na
                             end;
          if symb = '+' then begin symb := next;
                                   go to na
                             end
end;
symb := next;
S
```

This method is further discussed in Chapter 6, which describes how the parse of the original grammar can be constructed.

5.1 Bottom-to-top Methods

Consider the process of bottom-to-top left-to-right parsing, in which the correct production to be used can always be determined by some means. Then at an intermediate stage, the left end of the sentence has been partially contracted to a sequence of classes and words, and the right end of the sentence remains unaltered. This may be written as

$$I_1\ I_2 \ldots . I_n\ W_j\ W_{j+1} \ldots . W_l$$

where I_k is either a word or a class and W_h is a word.

If the next production to be used is determined by I_1 to I_n and W

to W_{j+t-1}, then the language is said to belong to $LR(t)$. Floyd has described a parser for languages of $LR(1)$.

If the next production is determined by the last m I's and the first s W's, then the language is said to be bounded context (m,s), or to belong to $BC(m,s)$. Sometimes $BC(m,s)$ is defined in terms of the last m words in I. The parsers described below are for languages in $BC(1,1)$.

If the *a priori* rule for selecting the next production is quick, as is likely if it only involves the two items at the join of I and W, then this technique provides a fast parsing method.

5.2 Operator Precedence Grammars

A grammar is said to be an operator grammar if there is no production of the form

$$A \rightarrow \alpha B C \beta$$

where α and β are any sequences, possibly empty. Many useful grammars are operator grammars, or can easily be changed into this form. For example, although the Algol grammar as given is not an operator grammar, the changes to make it into one are trivial.

Let us define three possible relations between words belonging to the vocabulary of an operator grammar, $=$, $<$, and $>$. Between a pair of words, in general, any number of these relations or none of them, may hold.

If there is a production

$$A \rightarrow \alpha p q \beta$$

or

$$A \rightarrow \alpha p B q \beta$$

where α and β are any sequences, possibly empty, then we shall say

$$p = q$$

If there is a production

$$A \rightarrow \alpha B q \beta$$

and a derivation

$$B \rightarrow \tau p$$

or

$$B \rightarrow \tau p C$$

then we shall say

$$p > q$$

If there is a production

$$A \rightarrow \alpha\, p\, B\, \beta$$

and a derivation

$$B \rightarrow q\, \tau$$

or,

$$B \rightarrow C\, q\, \tau$$

then we shall say

$$p < q$$

If at most one of these relations holds between each pair of words, then the grammar is an operator precedence grammar.

This information is clearly helpful when parsing is being done. For suppose that p and q, in that order, are two neighbouring words in a partly reduced sentence, that is, suppose that they are separated only by class symbols and not by other terminals. Then they belong to the same prime phrase if and only if $p = q$. A prime phrase is a phrase containing no other phrase. If $p > q$ then p belongs to a phrase to which q does not belong, and the classes between p and q also belong to that phrase. Similarly, if $p < q$ then q belongs to a phrase to which p does not belong. Hence it is easy to determine the phrase structure of the sentence. This does not itself give the full parse tree, because the productions used are not immediately deducible, but they can usually be found quite easily when the structure is known. The productions of the form

$$A \rightarrow B$$

are not revealed in this phrase structure, and the structure shown is that of the productions which contain at least one word.

Consider

$$S \rightarrow T \mid S + T$$
$$T \rightarrow P \mid T \times P$$
$$P \rightarrow id \mid (S)$$

which is a grammar for expressions with $+$, $\times$ and round brackets.

The following set of relations holds:

	+	×	(	)	*id*
+	>	<	<	>	<
×	>	>	<	>	<
(	<	<	<	=	<
)	>	>	=	>	
id	>	>		>	

We will now parse

$$a + b \times (c + d)$$

using C to stand for unknown classes, and showing the formation of the tree of phrases:

$a + b \times (c + d)$ $\quad$ $a + b \times (c + d)$
$\quad > \; < > \; < < > \; < >$

$C + b \times (c + d)$ $\quad$ $a + b \times (c + d)$
$\quad\quad < > \; < < > \; < >$

$C + C \times (c + d)$ $\quad$ $a + b \times (c + d)$
$\quad\quad\quad < \; < < > \; < >$

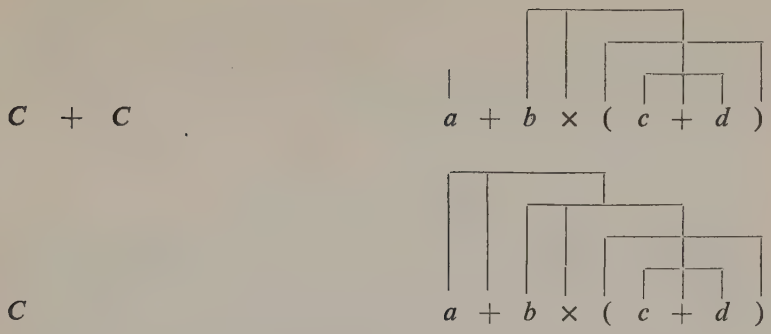
Note that the first relation is between + and ×; the C is ignored

$C + C \times (C + d)$ $\quad$ $a + b \times (c + d)$
$\quad\quad\quad < \; < \; < \; < >$

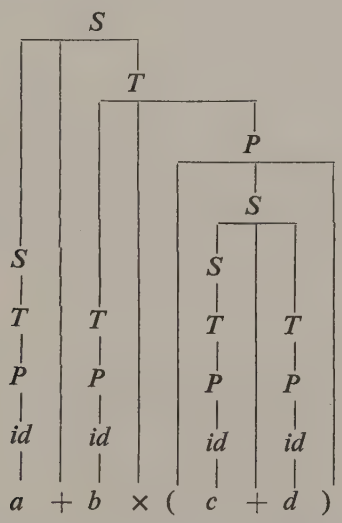
$C + C \times (C + C)$ $\quad$ $a + b \times (c + d)$
$\quad\quad\quad < \; < \; < \; >$

$C + C \times (C)$ $\quad$ $a + b \times (c + d)$
$\quad\quad\quad < \; < \; =$

$C + C \times C$ $\quad$ $a + b \times (c + d)$
$\quad\quad\quad <$



The fact that the analysis is really



has to be deduced by adding the effect of productions like

$$S \rightarrow T$$

and by finding the correct classes with which to label the nodes.

This may be described in a left-to-right mode as follows. A push-down list, I, of words and classes is formed, which at the start of parsing is empty. The first word of the sentence is placed in the push-down list initially. At an intermediate step the state is given by I and the rest of the sentence;

$$I_1 I_2 \ldots . I_n \quad W_j W_{j+1} \ldots . W_h$$

Consider the last word in I; let this be p. If $p = W_j$, then add W_j to the push-down list and proceed to the next word. If none of the relations $=$, $<$, $>$ holds between p and W_j, then the sentence is illegally formed. If $p > W_j$ then find the right-most set of words in I, having the relations

$$I_a < I_b = I_c = \ldots\ldots = p$$

All the words and classes from I_a to the end of the push-down list, but not including I_a, correspond to a production and may be contracted to a class. W_j is not removed from the sentence, and the process is repeated. The only difficulty lies in the fact that the class is not indicated by this process, but has to be found by some other means. Normally this is easy, since there is often some distinguishing word in the phrase. Thus, in the previous grammar, if a phrase containing $+$ is found, it can only arise from

$$S \rightarrow S + T$$

Note that this process entirely ignores the classes in the push-down list, and is controlled entirely by the relations between the words.

This algorithm can easily be written down if we ignore the formation of the parse tree:

```
I := NIL;
n3:symb := next;
n1:if I = NIL or eq(hd(I), symb) or less(hd(I), symb)
    then I := cons (symb, I)
    else if gr(hd(I), symb)
    then begin W := hd(I);
          n2:I   := tl(I);
              if I = NIL or less(hd(I), W)
              then go to n1
              else go to n2
    end
    else fail;
    go to n3
```

The relations between the words can be held in the form of a matrix, but often it is possible to define two weights, f and g, for each word so that

$$f(p) = g(q) \text{ is implied by } p = q$$
$$f(p) < g(q) \text{ is implied by } p < q$$
$$f(p) > g(q) \text{ is implied by } p > q$$

For the previous example, we can choose

	f	g
$+$	3	2
$\times$	5	4
(	1	6
)	6	1
id	5	6

In this case the expression

$$less(hd(I), symb)$$

in the algorithm can be replaced by

$$f(hd(I)) < g(symb)$$

and so on.

Floyd[11] gives algorithms for processing a grammar to determine if it is an operator precedence grammar, to find the matrix of relations, and to produce f and g if they exist. He also proves theorems to show that the method works.

To illustrate that f and g cannot always be found for an operator precedence grammar, consider

$$S \rightarrow a\ X \mid Y\ d$$
$$X \rightarrow b\ c\ b$$
$$Y \rightarrow c\ d\ a$$

for which the following relations hold, among others

$$a < b$$
$$c = b$$
$$a > d$$
$$c = d$$

An operator precedence grammar can be ambiguous. The parsing process only defines the tree and does not label the nodes.

While this way of parsing is extremely fast and is indeed widely used in compilers, especially for parsing arithmetic expressions, the set of grammars which it permits is rather too restricted. The chief difficulty is that the relations between the words have to be true universally, throughout the grammar. Thus in Algol

$$\textbf{else} > :$$

because in array declarations the ':' separates arithmetic expressions.

But also

$$\textbf{else} < :$$

because in conditional statements the ':' indicates a label. Hence Algol is not an operator precedence grammar. However, the contexts of the two occurrences are totally different, so the failure is disappointing. Of course a preliminary scan of the program could change the ':' in the array declaration into a new symbol.

5.3 Precedence Grammars

A kind of precedence grammar which is not restricted to operator grammars has been used by Wirth and Weber[33]. In this method the relations $=$, $<$ and $>$ are defined between pairs of items, both words and classes. In this case if there is a production

$$A \rightarrow \alpha\, I_1\, I_2\, \beta$$

we shall say that $I_1 = I_2$. If there is a production

$$A \rightarrow \alpha\, B\, I_2\, \beta$$

and a derivation

$$B \rightarrow \sigma\, I_1$$

or if there is a production

$$A \rightarrow \alpha\, B\, C\, \beta$$

and derivations

$$\begin{aligned} B &\rightarrow \sigma\, I_1 \\ C &\rightarrow I_2\, \delta \end{aligned}$$

then we shall say that $I_1 > I_2$:
If there is a production

$$A \rightarrow \alpha\, I_1\, B\, \beta$$

and a derivation

$$B \rightarrow I_2\, \sigma$$

then we shall say that $I_1 < I_2$. If at most one of these relations holds

between each pair of items, and if no two productions have the same right parts, the grammar is a precedence grammar.

Suppose that in a partly reduced sentence there are neighbouring items I_1 and I_2, in that order. They belong to the same prime phrase if and only if $I_1 \doteq I_2$. If $I_1 < I_2$ then I_2 belongs to a prime phrase to which I_1 does not belong. If $I_1 > I_2$ then I_1 belongs to a prime phrase to which I_2 does not belong. Just as for operator precedence grammars, so here it is easy to determine the phrase structure of the sentence. In this case, though, the parse can be constructed, because the productions are unique. A matrix of relations can be set up and used in a parser very similar to that for operator precedence grammars. Functions f and g that give a representation of the relations may also sometimes be constructed. Wirth and Weber give algorithms for generating the relations.

However, it is usually necessary to transform the grammar. For example, in

$$S \rightarrow T|S + T$$
$$T \rightarrow P|T \times P$$
$$P \rightarrow id|(\,S\,)$$

we have $+ = T$ from $S \rightarrow S + T$, and also $+ < T$ from $S \rightarrow S + T$ and $T \rightarrow T \times P$. There is also conflict in that $(= S$ and $(< S$ both hold. The grammar can be changed by adding extra classes

$$S \rightarrow U$$
$$U \rightarrow T|U + T$$
$$T \rightarrow V$$
$$V \rightarrow P|V \times P$$
$$P \rightarrow id|(\,S\,)$$

thus giving a precedence grammar.

Left-to-right parsing, using this method and ignoring the formation of the analysis tree is demonstrated below, using this modified grammar:

$$\begin{array}{cccccccc} a & + & b & \times & (& c & + & d &) \\ & > & & & & & & & \end{array}$$

$$\begin{array}{cccccccc} P & + & b & \times & (& c & + & d &) \\ & > & & & & & & & \end{array}$$

$$\begin{array}{cccccccc} V & + & b & \times & (& c & + & d &) \\ & > & & & & & & & \end{array}$$

$$\begin{array}{cccccccc} T & + & b & \times & (& c & + & d &) \\ & > & & & & & & & \end{array}$$

$$U + b \times (c + d) = < >$$

$$U + P \times (c + d) = < >$$

$$U + V \times (c + d) = < = < < >$$

$$U + V \times (P + d) = < = < < >$$

$$U + V \times (V + d) = < = < < >$$

$$U + V \times (T + d) = < = < < >$$

$$U + V \times (U + d) = < = < < = < >$$

$$U + V \times (U + P) = < = < < = < >$$

$$U + V \times (U + V) = < = < < = < >$$

$$U + V \times (U + T) = < = < < = = >$$

$$U + V \times (U) = < = < < >$$

$$U + V \times (S) = < = < = =$$

$$U + V \times P = < = =$$

$$U + V = <$$

$$U + T = =$$

$$U$$

$$S$$

This method appears a satisfactory one, though an adequate method of finding the classes corresponding to a production must be used.

5.4 Transition Matrices

This method of parsing, described by Samelson and Bauer[31], is very similar to that for operator precedence grammars. Again it is a bottom-to-top method. Two push-down lists are used. Into one of these the operators are placed, such as $+$ and $\times$. Into the other the operands are placed. Operators and operands are assumed to be distinguishable. The operator at the top of the push-down list and the next operator in the sentence are used to select from a matrix a piece of program to execute. This program usually either adds the new operator to the operator push-down list, or removes operators from the operator push-down list and operands from the operand list, combining them to form a new operand which is added to the operand list.

This is very close to the operator precedence method, the two push-down lists being a device for ignoring the classes. Methods for processing grammars to produce a matrix of routines have been described by Eichel *et al*[9].

6

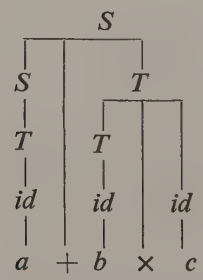
TRANSFORMATIONS ON GRAMMARS

We have seen various reasons for wishing to transform given grammars into other equivalent versions. For example, it may be necessary to remove left-recursion, or to put the grammar into normal form, or to change it into a precedence grammar. But the use of the original grammar is probably defined in terms of the parsing structure that would be generated from it, and not in terms of the parsing structure produced by the altered grammar. Hence we need a way of producing the original parse. The method of regenerating it described below has been used in a compiler writing system.[14]

We shall introduce a third kind of item and we shall call these items *actions*. They will be written in bold type. Actions are like words, in that they are terminal and give rise to no further substitution, but they are to be interpreted differently. A sentence generated by a grammar with actions in it will contain a mixture of words and actions, but a sentence presented for parsing will contain only words. During the matching process which generates the parse, the actions are ignored. For example.

$$S \rightarrow T \mid S + T$$
$$T \rightarrow id \mid T \times id$$

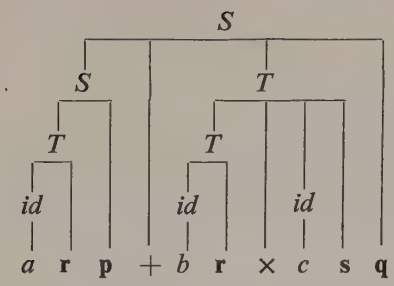
gives a parse



In the corresponding grammar with actions

$$S \rightarrow T\,\mathbf{p} \mid S + T\,\mathbf{q}$$
$$T \rightarrow id\,\mathbf{r} \mid T \times id\,\mathbf{s}$$

the parse is



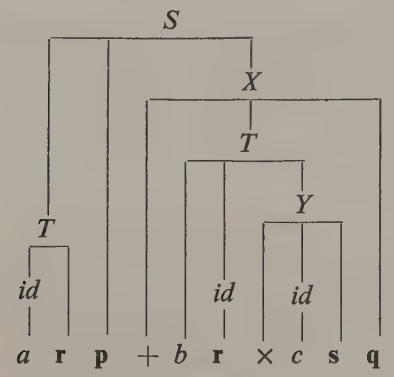
Since a different action has been put at the end of each production, if we were given the words and actions

$$a\,\mathbf{r}\,\mathbf{p} + b\,\mathbf{r} \times c\,\mathbf{s}\,\mathbf{q}$$

it would be easy to generate the parse tree. As the sentence is scanned from left to right, each time an action is reached the corresponding production is used on the items to the left. But there is no compulsion to use different actions in different productions, nor to place the actions at the end of the productions. If, when a transformation is to be made on a grammar, it is made instead on the same grammar with actions at the end of the productions, then we get a new grammar with the same actions in different places. For example, the grammar above can be transformed into

$$\begin{aligned} S &\rightarrow T\,\mathbf{p} \mid T\,\mathbf{p}\,X \\ X &\rightarrow + T\,\mathbf{q} \mid + T\,\mathbf{q}\,X \\ T &\rightarrow id\,\mathbf{r} \mid id\,\mathbf{r}\,Y \\ Y &\rightarrow \times id\,\mathbf{s} \mid \times id\,\mathbf{s}\,Y \end{aligned}$$

Using this grammar to parse the same sentence as before we get



which is quite a different parse structure, but the actions occur in the same place as before. Hence by using the actions of the transformed grammar we can regenerate the original parse tree.

Note that in this case we turned a left-recursive grammar into one which was not left-recursive.

Chapter 7 shows that the compilation process can often be described by means of actions in the grammar, which act on a stack of results. There is no need to generate the tree, but the actions can be applied directly in the transformed grammar to do the compilation.

6.1 Transformations to remove Left-recursion

It has been shown by Greibach that, given a grammar, there always exists an equivalent grammar which is not left-recursive. Various transformations have been described which achieve this.[14, 19, 26, 35] The following description is of one such transformation.[14]

We shall use ϕ to stand for the empty phrase, and ϵ to stand for an illegal phrase. If ϕ occurs in a sentence it may be removed: if ϵ occurs in a sentence, the sentence is illegal. There is an obvious relationship between ϕ and ϵ on the one hand, and 1 and 0 on the other. The symbol δ_{rs} will be used to stand for ϕ if r is equal to s, and to stand for ϵ if r is not equal to s. Consider first a very simple left-recursive definition

$$X \rightarrow a | X\, b$$

which defines sentences that start with an a and then contain any number of b's. The grammar

$$X \rightarrow a\ Y$$
$$Y \rightarrow \phi | b\ Y$$

defines the same sentences. Here Y is an auxiliary class invented for the transformation. The general case is based on the same construction.

Suppose that there are n classes, L_i, all involved in a left-recursive cycle. That is for each L_i, L_j there is a derivation $L_i \rightarrow L_j\, \alpha$. If $i=j$ then α must not be ϕ, but otherwise it may be. We shall use a repeated suffix convention:

$$L_i \rightarrow A_i | L_j\, B_{ji}$$

which means

$$L_1 \rightarrow A_1 | L_1\, B_{11} | L_2\, B_{21} | \ldots | L_n\, B_{n1}$$

$$L_2 \rightarrow A_2 | L_1\, B_{12} | L_2\, B_{22} | \ldots | L_n\, B_{n2}$$

$$\vdots$$

$$L_n \rightarrow A_n | L_1\, B_{1n} | L_2\, B_{2n} | \ldots | L_n\, B_{nn}$$

Collect all the productions for the L_i, and define new auxiliary classes as necessary to put the productions in the form

$$L_i \rightarrow A_i | L_j B_{ji}$$

where the A_i cannot start with any L_j; there is no derivation $A_i \rightarrow L_j \alpha$. It may be necessary that the productions for the A_i or B_{ji} should contain ϕ or ϵ.

Now an equivalent set of productions for the L_i with no left-recursive classes is given by

$$L_i \rightarrow A_i X_{ji}$$
$$X_{rs} \rightarrow \delta_{rs} | B_{rt} X_{ts}$$

where the X_{rs} are n^2 auxiliary classes.

First, we notice that any phrase derived from L_i in the first grammar is also to be derived from L_i in the second grammar and vice versa. For the phrases derived from L_i in the first grammar are of the form,

$$A_p B_{pq} B_{qr} \ldots B_{zi} \text{ (no repeated suffix convention)}$$

and so are those of the second. Moreover, if the B_{ij} and A_i are all distinct, then both grammars are unambiguous. Hence, if they are not distinct, and there are d derivations of a phrase in one grammar, there are also d derivations of it in the other grammar.

Second, there can be no derivation

$$X_{ij} \rightarrow X_{ij}$$

since such a derivation can only arise if there is a derivation

$$B_{ip} B_{pq} B_{qr} B_{zi} \rightarrow \phi$$

This cannot be true, since there would be a derivation $L_i \rightarrow L_i$ in the first grammar. A derivation $L_i \rightarrow L_i$ in the second grammar would also imply the same derivation in the first, which is not so.

Third, the second grammar may contain productions involving ϵ. We can remove these productions without leaving any classes which have no terminal derivations. In the first grammar there is a derivation

$$L_i \rightarrow L_j \alpha$$

for each pair i, j. Hence for each i, j there is a set of indices such that

$$B_{jp} B_{pq} \ldots B_{zi}$$

has a terminal derivation. Therefore for each X_{rs}, there is a derivation

$$X_{rs} \rightarrow B_{rh}\, B_{hk} \ldots B_{ms}\, X_{ss}$$

and since $X_{ss} \rightarrow \phi$, it follows that X_{rs} has a legal terminal derivation.

Hence the grammars are equivalent. Actions can be used to regenerate the parse according to the first grammar because

$$L_i \rightarrow A_i\, \mathbf{p}_i | L_i\, B_{ji}\, \mathbf{q}_{ji}$$

transforms into

$$\begin{aligned} L_i &| A_j\, \mathbf{p}_j\, X_{ji} \\ X_{rs} &\rightarrow \delta_{rs} | B_{rt}\, \mathbf{q}_{rt}\, X_{ts} \end{aligned}$$

By successively removing the left-recursive rules in this fashion, the grammar can be altered until it is free from left-recursion.

6.2 Other Transformations

In both the top-to-bottom and the bottom-to-top methods, greater efficiency is likely if the productions for each class start, as far as possible, with disjoint sets of words. The *a priori* rules which use the set of possible starting words for the productions will obviously cut out more productions in this case. The limit is reached, as described in Chapter 5, when for each class the sets of words starting each production are disjoint. In this case, the top-to-bottom and bottom-to-top methods merge into one, and the analysis is very fast. It is therefore useful to consider transformations which produce this effect.

We can apply to productions distributive transformations, just like the distributive transformations of $+$ and $\times$. The grammar

$$\begin{aligned} X &\rightarrow Y\, Z \\ Y &\rightarrow A | B \end{aligned}$$

is equivalent to

$$X \rightarrow A\, Z | B\, Z$$

and

$$X \rightarrow C\, D | C\, E$$

is equivalent to

$$\begin{aligned} X &\rightarrow C\, Y \\ Y &\rightarrow D | E \end{aligned}$$

By repeatedly applying these transformations to a given grammar, we can usually decrease the overlap of productions, and often eliminate it.[14] For if we have two productions for a class which start with identical items,

$$X \rightarrow A\ B_1\ B_2 \ldots | A\ C_1\ C_2 \ldots$$

we can invent a new class and write

$$X \rightarrow A\ Y$$
$$Y \rightarrow B_1\ B_2 \ldots | C_1\ C_2 \ldots$$

If we have two productions for a class which are not identical, but which can start with the same word, for example

$$X \rightarrow A\ B_1\ B_2 \ldots | C\ D_1\ D_2 \ldots$$
$$A \rightarrow P | Q$$
$$C \rightarrow P | R$$

we can first write

$$X \rightarrow P\ B_1\ B_2 \ldots | Q\ B_1\ B_2 \ldots | P\ D_1\ D_2 . | R\ D_1\ D_2 .$$

and then

$$X \rightarrow P\ Y | Q\ B_1\ B_2 \ldots | R\ D_1\ D_2 \ldots$$
$$Y \rightarrow B_1\ B_2 \ldots | D_1\ D_2 \ldots$$

Unfortunately, there are two difficulties which prevent this method from always eliminating the overlaps. The first is that it is necessary to use ϕ, which thereby introduces into consideration the words which follow a class. Given

$$X \rightarrow A | A\ B$$
$$B \rightarrow X\ C$$
$$C \rightarrow B | D$$

we produce

$$X \rightarrow A\ Z$$
$$Z \rightarrow \phi | B$$
$$Y \rightarrow X\ C$$
$$C \rightarrow B | D$$

and now trouble arises with

$$Z \rightarrow \phi | B$$

since Z can be followed by B.

The second trouble arises because the process may get into an infinite loop. Consider

$$A \rightarrow X|Y$$
$$X \rightarrow B|L\ X$$
$$Y \rightarrow C|L\ Y$$

which first gives

$$A \rightarrow B|C|L\ X|L\ Y$$
$$X \rightarrow B|L\ X$$
$$Y \rightarrow C|L\ Y$$

and then

$$A \rightarrow B|C|L\ Z$$
$$Z \rightarrow X|Y$$
$$X \rightarrow B|L\ X$$
$$Y \rightarrow C|L\ Y$$

Now we are in the same position with respect to Z, X and Y as we were originally with A, X and Y, so the process continues for ever. In this case we would notice the similarity and write

$$A \rightarrow B|C|L\ A$$
$$X \rightarrow B|L\ X$$
$$Y \rightarrow C|L\ Y$$

Another example of failure is

$$S \rightarrow a|a\ S\ a$$

The transformations which have been used to produce operator precedence grammars or precedence grammars are relatively simple. They usually involve eliminating a non-recursive class, or introducing new classes as shown in Chapter 5.

UNIVERSITY OF VICTORIA LIBRARY

7

USING GRAMMATICAL ANALYSES FOR COMPILATION

This book is primarily concerned with analysing sentences, rather than with the subsequent processing of the analyses. There are a number of papers which may be consulted about this.[4, 6, 10, 20, 21, 28, 30, 31] It is possible to wait until the whole analysis tree is formed and then process it. Alternatively, the compiling actions may be performed as the analysis goes on. A brief indication of a way of doing this is given below.

Consider a grammar with actions at the ends of the productions. During the left-to-right analysis a push-down list is formed. The actions either add something to this push-down list or take some items off it, operate on them, and put a result back on the push-down list. Suppose as a trivial example, that we wish to turn expressions like

$$a + b + c + d$$

into

LD, a
ADD, b
ADD, c
ADD, d

We shall use the grammar

$$S \rightarrow id\ \mathbf{p} | S + id\ \mathbf{q}$$

The representative *id* is assumed to add the name of the identifier to the push-down list. The action **p** is taken after the first identifier so

p means $x := pull; push\ (concatenate\ (\text{'}LD,\text{'}, x))$

The action **q** is taken when the top two items of the push-down list result from *S* and *id* so

q means $x := pull; y := pull; push\ (concatenate(y, concatenate(\text{'}ADD,\text{'}, x)))$

We shall use this grammar in the transformed version

$$S \rightarrow id\ \mathbf{p}\ X$$
$$X \rightarrow \phi | + id\ \mathbf{q}\ X$$

The following are the successive steps:

SENTENCE	ACTION USED	PUSH-DOWN LIST
$a+b+c+d$	–	–
$+b+c+d$	*id*	('*a*')
$+b+c+d$	**p**	('*LD*, *a*')
$+c+d$	*id*	('*b*', '*LD*, *a*')
$+c+d$	**q**	('*LD*, *a ADD*, *b*')
$+d$	*id*	('*c*', '*LD*, *a ADD*, *b*')
$+d$	**q**	('*LD*, *a ADD*, *b ADD*, *c*')
–	*id*	('*d*', '*LD*, *a ADD*, *b ADD*, *c*')
–	**q**	('*LD*, *a ADD*, *b ADD*, *c ADD*, *d*')

If on the other hand we had wanted

$$a + b + c + d$$

to give

LD, *d*
ADD, *c*
ADD, *b*
ADD, *a*

then we would use

$$S \rightarrow id\ \mathbf{p} | id + S\ \mathbf{r}$$

in which *id* and **p** have the same effect as before. However, when **r** is done the top two items of the push-down list result from *id* and *S*, so

r means *x* := *pull*; *y* := *pull*; *push*(*concatenate*(*x*, *concatenate*('*ADD*,' *y*)))

Now the sequence of steps is:

SENTENCE	ACTION USED	PUSH-DOWN LIST
$a+b+c+d$	–	–
$+b+c+d$	*id*	('*a*')
$+c+d$	*id*	('*b*', '*a*')
$+d$	*id*	('*c*', '*b*', '*a*')
–	*id*	('*d*', '*c*', '*b*', '*a*')
–	**p**	('*LD*, *d*', '*c*', '*b*', '*a*')
–	**r**	('*LD*, *d ADD*, *c*', '*b*', '*a*')
–	**r**	('*LD*, *d ADD*, *c ADD*, *b*', '*a*')
–	**r**	('*LD*, *d ADD*, *c ADD*, *b ADD*, *a*')

As a more elaborate example consider

$$S \rightarrow id := E;$$
$$E \rightarrow T | E + T | E - T$$
$$T \rightarrow id | T \times id \mid T / id$$

We can introduce compiling actions

$$S \rightarrow id := E; \mathbf{p}$$
$$E \rightarrow T | E + T \mathbf{q} | E - T \mathbf{r}$$
$$T \rightarrow id \mathbf{s} | T \times id \mathbf{t} | T / id \mathbf{u}$$

where
p means *push* (*concatenate*(*pull*, *concatenate*('*ST*,', *pull*)))
q means *push*(*concatenate*(*pull*, *concatenate*('*ST*,*W1*',
concatenate(*pull*, '*ADD*, *W1*'))))
r means *push*(*concatenate*(*pull*, *concatenate*('*ST*, *W1*',
concatenate(*pull*,*NEGADD*, *W1*'))))
s means *push*(*concatenate*('*LD*,', *pull*))
t means $z :=$ *pull*; *push*(*concatenate*(*pull*, *concatenate*('*MULT*,', *z*)))
u means $z :=$ *pull*; *push*(*concatenate*(*pull*, *concatenate*('*DIV*,', *z*)))

For example this will translate

$$g := a + b \times c / d + e \times f$$

into

LD, *e*
MULT, *f*
ST, *W1*
LD, *b*
MULT, *c*
DIV, *d*
ADD, *W1*
ST, *W1*
LD, *a*
ADD, *W1*
ST, *g*

We transform the grammar giving

$$S \rightarrow id := E \mathbf{p}$$
$$E \rightarrow T X$$
$$X \rightarrow \phi | + T \mathbf{q} X | - T \mathbf{r} X$$
$$T \rightarrow id \mathbf{s} Y$$
$$Y \rightarrow \phi | \times id \mathbf{t} Y | / id \mathbf{u} Y$$

This can be translated as in the first method of Chapter 5, into the program.

```
procedure S;
begin if not id(symb) then fail;
      symb := next;
      if symb ≠ ':' then fail;
      symb := next;
      if symb ≠ '=' then fail;
      symb := next;
      E;
      if symb ≠ ';' then fail;
      p; comment a call of the action for p;
end;
procedure E;
begin T;
   na: if symb = '+' then begin symb := next
                                T;
                                q;
                                go to na
                          end
       else if symb = '-' then begin symb := next;
                                     T;
                                     r;
                                     go to na
                               end
end;
procedure T;
begin if not id(symb) then fail;
      s;
   nb: if symb = '×' then begin symb := next;
                                if not id(symb) then fail;
                                t;
                                go to nb
                          end
       else if symb = '/' then begin symb := next;
                                     if not id(symb) then fail;
                                     u;
                                     go to nb
                               end
end;
symb := next;
S
```

This program performs the required function and the process of translation of the grammar into the program is quite automatic.

Real compilers have much more complex tasks, but as far as the analysis phase is concerned, these techniques are usually adequate.

7.1 Illegal Sentences

If a parser is presented with a sentence which does not belong to the grammar which is being used, it will of course detect the error. But we wish to indicate the error in a comprehensible way, and we may wish to carry on the parse to find later errors. It cannot be said that any wholly satisfactory methods of doing this in general have been described.

If the parser is one which parses the sentence one word at a time from left to right, then it is possible to indicate the place in the sentence at which the final failure occurred. But a human will not always agree about the position of the error. For example, an omitted open bracket in an arithmetic expression will not show up as an error until the end of the expression. However, an indication of the place where the analysis failed and preferably a word which would have enabled the parser to carry on, is usually sufficient. In the case of a parser which explores each parse until it fails and then back-tracks to pick up another parse, the error indication may be more difficult to provide. An indication of the place where the longest parse failed could be given.

The problem of picking up the analysis and trying to carry on is more difficult, and general methods are unlikely to be universally useful. Most compilers move on to a recognisable place, such as the end of a statement, and carry on from there. The recognition of such a place is likely to be *ad hoc*.

There is a possibility, for parsers which simultaneously carry on a number of analyses, that several assumptions can be made about the error and the several parses resulting used to continue. For example, it might be assumed that a word was ommitted at the error detection point, or that a wrong word was inserted, or that a wrong word was substituted. Such methods have been described, but though they work in simple cases, in more complex ones the results are not always good.

APPENDIX 1

ELEMENTARY LIST PROCESSING

The parsing programs in this book have been written using list processing techniques. This is because the demands of the programs for space are unpredictable, and because the grammars and analyses are highly suitable for representation as lists. Clearly these programs can be turned into other variants using, for example, arrays to represent the grammars. A brief account is given below of enough list processing to understand the programs.

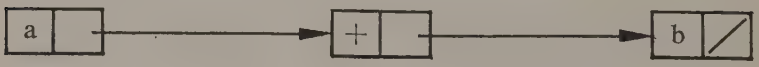
A certain area in the computer store is set aside to hold lists, and this area is divided up into cells, each of which is sufficiently large to hold two items. The items that can be put into these cells are either the addresses of other cells, or pieces of data called atoms. The atoms are so called because their structure is not the concern of the list processing system. We could write down the contents of these cells and give their explicit addresses, but such a notation is much too cumbersome for practical use. Instead we shall use one of two paper notations. In the first, the cell containing two items is represented by a box



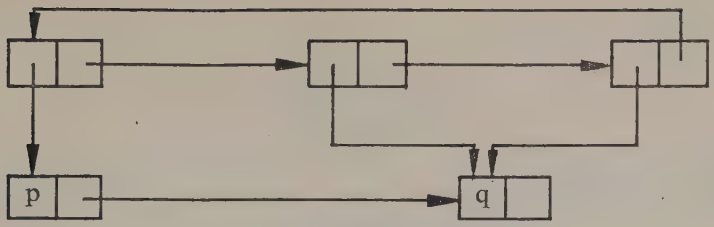
and the cell items which are addresses of other cells are represented by arrows pointing to those cells



The atoms are represented by writing them into the boxes



The oblique stroke represents a special atom called *NIL*. Structures of arbitrary complexity can be built up

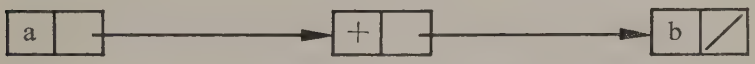


Note that the structures can be circular, and there can be any number of pointers to the same cell.

By convention, the left-hand member of the box is called the head, and the right-hand member is called the tail. The alternative notation is to write

$$(a, +, b)$$

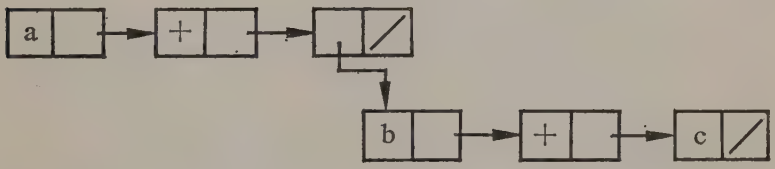
for the list



The items of the list are those in the heads, and the tails are used to string together the cells forming the list. Similarly

$$(a, +, (b, +, c))$$

represents



Programs are written in Algol, to which variables having lists as values have been added. Three functions are used to perform the primitive list processing operations. If x is a list, the value of $hd(x)$ is the value of the item in its head part. Similarly, $tl(x)$ gives the item in the tail part of x. The function cons(x, y) gives as its value a cell containing x in its head part and y in its tail part. Note that to form the list

$$(a, b, c)$$

we write

$$cons(a, cons(b, cons(c, NIL)))$$

Note also that while

$$hd(cons(x,y)) = x$$
$$tl(cons(x, y)) = y$$

it is not true that

$$cons(hd(x), tl(x)) = x$$

because *cons* creates a new cell. The value of $cons(hd(x), tl(x))$ is a cell which is a copy of x, it is not x itself.

As an example consider a program to reverse a list m, putting the result into n:

```
n := NIL;
r: if m ≠ NIL then begin n := cons(hd(m), n);
                         m := tl(m);
                         go to r
                   end
```

APPENDIX 2

AN ALGORITHM FOR TOP-TO-BOTTOM PARSING

The functions *hd*, *tl* and *cons* are described in Appendix 1. The function *next* reads the next word of the sentence, and *terminal* is a function which is true if its argument is a word and false if its argument is a class. The function *apriori* is true if its argument, which is a production, satisfies the *a priori* rules.

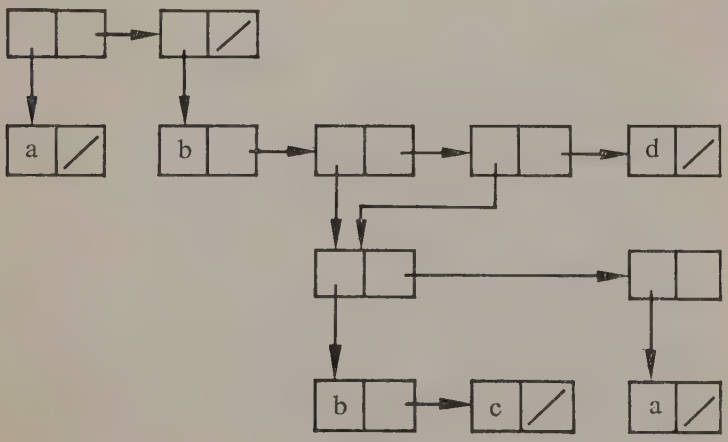
Inside the computer, the words are represented in some fashion which we shall indicate by enclosing them in quotation marks. The classes are represented by a list of their alternative productions, and the productions are represented as a list of their constituents in sequence. The special class which defines sentences is placed in the variable *sentence*. So for example in the grammar

$$S \rightarrow a|b\ T\ T\ d$$
$$T \rightarrow b\ c|a$$

the variable *sentence* contains the list

(('*a*'), ('*b*', (('*b*', '*c*'), ('*a*'))), (('*b*', '*c*'), ('*a*')), ('*d*'))

In fact, of course, the two occurrences of *T* will be pointers to the same sub-list:



Interesting grammars, which contain recursive classes, will have circular lists to represent the classes.

In the program, the variable *la* is used to hold a list of all the partial parses currently active, and the variable *symb* holds the current word. As a result of examining the current word and the partial parses, some of the parses fail and are forgotten, some pass into *lb*, which holds the partial parses ready for the next word, and some produce further parses which are put back into *la* and processed again. When *la* is empty, *lb* is transferred into it and the process is repeated. The parses which are complete at the end of the sentence are the correct parses.

Each partial parse is represented by a list of the ends of productions. It indicates the structure which the rest of the sentence must have for it to conform with this parse. So if we are analysing *b a b c d* in

$$S \rightarrow a|b\ T\ T\ d$$
$$T \rightarrow b\ c|\ a$$

and we have just processed *b a b*, the remaining partial parse will be

$$((\text{'}c\text{'}), (\text{'}d\text{'}))$$

indicating that what remains of the sentence must be a *c* followed by a *d*. The

$$(\text{'}c\text{'})$$

is the remainder of the production $T \rightarrow b\ c$ and the

$$(\text{'}d\text{'})$$

is the remainder of $S \rightarrow b\ T\ T\ d$.

```
    la := cons(cons(cons(sentence, NIL), NIL), NIL);
z4:lb := NIL; symb := next;
zz:if la≠NIL then
    begin lc := hd(la); la := tl(la);
      z1:if lc = NIL then begin if lastword then success
                                              else go to zz;
      ld := hd(lc); lc := tl(lc);
      if ld = NIL then go to z1;
      le := hd(ld); ld := tl(ld);
      if terminal (le) then begin if le = symb
                                  then lb := cons(cons(ld,lc), lb);
                                  go to zz
                        end;
```

```
        z3:if le ≠ NIL then
            begin if not apriori(hd(le)) then go to z3;
                  la := cons(cons(hd(le), cons(ld,le)), la);
                  le := tl(le);
                  go to z3
            end;
            go to z2
end;
if lb = NIL then fail;
la := lb;
go to z4;
```

REFERENCES

Abbreviations: *ACM* Association for Computing Machinery
AFIPS American Federation of Information Processing Societies
CACM Communications of the Association for Computing Machinery
IEEE Institute of Electrical and Electronic Engineers
IFIP International Federation for Information Processing
JACM Journal of the Association for Computing Machinery
SJCC Spring Joint Computer Conference

1. Bar Hillel, Y. 'Language' and information.' Addison-Wesley, (1964).
2. Brooker, R. A. *CACM*, Vol. 10, p. 223 (April 1967).
3. Brooker, R. A. and Morris, D. 'An assembly program for a phrase structure language.' *Computer Journal*, Vol. 3, p. 168 (1960).
4. Brooker, R. A. and Morris, D. 'A general translation program for phrase structure languages.' *JACM*, Vol. 9, p. 1 (Jan. 1962).
5. Brooker, R. A., Morris, D. and Rohl, J. S. 'Experience with the compiler-compiler.' *Computer Journal*, Vol. 9, p. 345 (1967).
6. Cheatham, T. E. and Satley, K. 'Syntax directed compiling.' *Proc. AFIPS*, 1964 p. 31 (SJCC).
7. Chomsky, N. 'Formal properties of grammars.' *Handbook of Mathematical Psychology*, Vol. 2, p. 323, Wiley (1963).
8. Chomsky, N. 'Aspects of the theory of syntax.' MIT Press (1965).
9. Eichel, J., Paul, M., Bauer, F. L. and Samelson, K. 'A syntax controlled generator of formal language processors'. *CACM*, Vol. 6, p. 451 (Aug. 1963).
10. Feldman, J. and Gries, D. 'Translator writing systems.' *CACM*, Vol. 11, p. 77 (Feb. 1968).
11. Floyd, R. W. 'Syntactic analysis and operator procedence.' *JACM*, Vol. 10, p. 316 (July 1963).
12. Floyd, R. W. 'Bounded context syntactic analysis.' *CACM*, Vol. 7, p. 62 (Feb. 1964).
13. Floyd, R. W. 'The syntax of programming languages—a survey.' *IEEE Trans*, *EC*13, Vol. 4, p. 346 (Aug. 1964).
14. Foster, J. M. 'A syntax improving program.' *Computer Journal*, Vol. 11, p. 31 (May 1968).
15. Ginsburg, S. 'The mathematical theory of context-free languages.' McGraw-Hill Inc. (1966).
16. Ginsburg, S. and Greibach, S. 'Deterministic context-free languages.' *Information and Control*, Vol. 9, p. 620 (1966).

17. Greibach, S. 'Formal Parsing Systems.' *CACM*, Vol. 7, p. 499 (Aug. 1964).
18. Griffiths, T. V. and Petrick, S. R. 'On the relative efficiency of context-free grammar recognisers.' *CACM*, Col. 8, p. 289 (May 1965).
19. Griffiths, T. V. and Petrick, S. R. 'Top-down versus bottom-up analysis.' *IFIP Congress* 68, B80.
20. Ingerman, P. Z. 'A syntax oriented translator.' Academic Press (1966).
21. Irons, E. T. 'A syntax directed compiler for Algol 60.' *CACM*, Vol. 4, p. 51 (Jan. 61).
22. Irons, E. T. 'The structure of the syntax-directed compiler.' *Annual Review in Automatic Programming*, Vol. 3, p. 207 (1963).
23. Irons, E. T. 'An error correcting parse algorithm.' *CACM*, Vol. 6, p. 669 (Nov. 1965).
24. Kuno, S. and Oettinger, A. G. 'Multiple path syntactic analyser.' *IFIP Congress* 62, p. 306.
25. Kuno, S. 'The predictive analyser and a path elimination technique.' *CACM*, Vol. 8, p. 453 (July 1965).
26. Kurki-Suonio, R. 'On top-to-bottom recognition and left-recursion.' *CACM*, Vol. 9, p. 527 (July 1966).
27. Landweber, P. S. 'Decision problems of phrase structure grammars.' *IEEE Trans. EC*13, p. 354 (Aug. 1964).
28. Ledley, R. S. and Wilson, J. B. 'Automatic programming language translation through syntactical analysis.' *CACM*, Vol. 5, p. 145 (March 1962).
29. Narasimhan, R. 'Syntax directed translation of classes of pictures.' *CACM*, Vol. 9, p. 166 (March 1966).
30. Reynolds, J. C. 'An introduction to the COGENT programming system.' ACM 20th *National Conference* 1965 p. 422.
31. Samelson, K. and Bauer, F. L. Sequential formula translation. *CACM*, Vol. 3, p. 76 (Feb. 1960).
32. Unger, S. H. 'A global parser for context-free phrase structure grammars.' *CACM*, Vol. 11, p. 240 (April 1968).
33. Wirth, N. and Weber, H. 'EULER—a generalisation of Algol, and its formal definition.' Part 1, *CACM*, Vol. 9, p. 13 (Jan. 1966); Part 2, *CACM*, Vol. 9, p. 89 (Feb. 1966).
34. Amarel, S. To be published.

INDEX

action, 44

class, 5
—left recursive, 9, 46
—right recursive, 9
—self embedding, 10
cons, 57

derivation, 6

grammar,
—ambiguous, 7
—context-free, 2, 6
—equivalent, 22
—operator precedence, 34
—precedence, 40
—transformations on, 46

hd, 57

language, natural, 2, 24
list,
—pushdown, 37
—processing, 56

NIL, 56

parse, 2
—tree, 18
parser, 2
—bottom-to-top, 17, 24
—global, 28
—top-to-bottom, 17, 18
phrase, 6
production, 5

representative, 8

sentence, 2, 5
syntax,
—analyser, 2
—errors, 55

terminal, 6
test,
—*a priori*, 14
—bottom-to-top 26
—top-to-bottom 22
tl, 57

word, 2, 5

DATE DUE

JAN 17 1980	JAN 25 1980		
FEB 15 1980	MAR 25 1980		
AUG 12 1981			
AUG 5 1981	AUG - 4 1981		
	JAN 2 1982		
OCT 8 1981			
JUL 20 1982	AUG 25 1982		
JUL 5 1983	OCT 29 1983		
OCT 15 1986	DEC 20 1986		
MAR 1 1988	FEB 23 1988		

261-2500

Printed in USA

QA76.5
F61

Foster, John M.

Automatic syntactic analysis.

O7-BPI-670